

Software Requirements Specification

Introduction to Software Engineering (15031001) · Week 3 · Requirements & Specification

Field	Value
Team name	MFU 888
Project title	Shop AI - The Digital Waiter for Restaurants
SDLC chosen in M1 + reason	Agile (iterative sprints): the AI's safety answers and its Burmese replies needed repeated test-and-fix rounds (a 30-question test set and 4 reviews by a native Burmese speaker), and the screens are refined from user feedback every sprint.
Date / version	v1.1 October 2026
GitHub repo	https://github.com/Aung-Nyi-Thant/WaiterAi

1. Introduction

1.1 Purpose and scope

Restaurant menus reached by QR code are disliked by diners: 90% of US diners prefer a printed menu (US Foods Diner Dispatch 2024) and 26% say the text is too small (Toast survey); most QR menus are scanned PDFs with no way to ask a question. Tourists and migrant workers often cannot read the menu language, and allergy questions are risky because a wrong chatbot answer can create legal liability (Moffatt v. Air Canada, 2024). Small restaurants cannot afford large systems. Shop AI is a web application in which a diner scans a table QR code, browses a photo menu and chats with an AI waiter in Thai, Burmese or English, without an account. Safety-critical answers (allergens, vegetarian lists, prices, opening hours, sold-out dishes) are built from the restaurant's own database with fixed sentences; the AI model runs locally on the restaurant's computer, so no customer data goes to a cloud AI service. Owners manage the menu and allergen data, import a menu from a photo and see what diners asked; waiters and chefs see live calls and orders. Success looks like a diner receiving a correct allergy, price or opening-hours answer in their own language within 15 seconds, and an owner publishing a first menu within 30 minutes.

1.2 Users and stakeholders

Name / role	Kind	What they need	What they fear / forbid
Diner (tourist, migrant worker, local customer)	primary user	A readable menu with photos in Thai, Burmese or English; quick answers about allergens, prices and vegetarian food; no app or login	Wrong allergy information; not being understood; being forced to install an app
Restaurant owner	primary user	Cheap, fast setup (menu from a photo in under 30 minutes); control of allergen data, hours and sold-out dishes; insight into what diners ask	Cost, setup effort, an AI that invents prices or ingredients, legal blame for AI mistakes
Waiter	primary user	A live list of table calls and the dishes each table picked, with allergy warnings	Missing a call or an allergy note; extra work per order
Chef	primary user	A live list of table calls and the dishes each table picked, with allergy warnings	Cooking a dish that contains a diner's allergen; a cluttered screen

Name / role	Kind	What they need	What they fear / forbid
Academic evaluator	assessment	Traceable requirements, quantified non-functional requirements, tested AI behaviour	Unchecked AI hallucinations, vague requirements
Food-safety data-protection authority (EU)	regulatory body	Allergen information shown clearly	Allergen data invented by AI;
1169/2011 allergen rules, PDPA/GDPR)		Allergen information shown clearly	personal data leaking to third parties

2. Overall description

2.1 Product context — what it does / does not

Shop AI gives each restaurant a QR-code web menu with a menu-aware AI waiter (Thai, Burmese, English), a restaurant back office, and live screens for waiters and chefs. Everything below represents the complete scope for this term - nothing else is implied.

IN SCOPE (MVP this term)	OUT OF SCOPE (explicit promise)
• FR-1 Multilingual menu browsing • FR-2 Safe AI waiter chat • FR-3 Picks, orders and staff calls • FR-4 Waiter and kitchen live screens • FR-5 Owner account and menu / allergen management • FR-6 Menu import from a photo • FR-7 Restaurant settings (hours, specials, FAQ, AI voice) • FR-8 QR codes per table • FR-9 Staff accounts (PIN) • FR-10 Insights	• Online payment or bill splitting • Delivery and loyalty programmes • LINE / WhatsApp channels • POS integration • Native mobile apps • Billing plans • AI-generated dish photos • Order status shown to diners • Owner dashboard translated into Thai / Burmese

2.2 Assumptions and constraints

- **Assumption:** The server computer (tested on a MacBook Air M4, 32 GB memory) is on while the restaurant is open, runs Node.js 22.13 or newer and Ollama with the model gemma4:12b (about 8.4 GB).
- **Assumption:** The owner enters or confirms allergen data; the system never infers allergens from a photo or a dish name.
- **Constraint:** No customer data is sent to a cloud AI service; all AI runs locally through Ollama.
- **Constraint:** Allergen, price, hours and availability answers must come from the database. The AI must never state that a dish is "safe" or "allergy-free".
- **Constraint:** Diners must not need an account or an app. Languages: Thai, Burmese (Unicode), English.
- **Constraint:** One server computer processes one AI answer at a time; photo uploads are limited to JPG, PNG and WebP under 8 MB.

3. Functional requirements

Format: As a [role], I want [goal], so that [benefit]. The pains below come from published surveys in our market research report and from three M1 interviews (22 Sep 2026); checking the requirements with a stakeholder is still to be done (see Section 9). Every acceptance criterion is testable.

FR-1 - Multilingual Menu Browsing

- **User story:** As a diner, I want to open the restaurant menu by scanning a QR code and switch between Thai, Burmese and English, so that I can read the menu in my own language without installing an app.
- **Pain this traces to:** 90% of diners dislike current QR menus, 26% find the text too small, 66% want photos, and about 71% of tourists prefer restaurants that translate their menu.
- Given a diner opens /r/[restaurant]?t=[table] on the same network, when the page loads, then dishes with name, price, tags and up to three allergens are shown within 3 seconds and no login is requested.
- Given the diner taps TH, MY or EN, when the language changes, then all labels and dish names change within 1 second without reloading; and given a dish is switched to sold out, then it shows "sold out today" within 20 seconds.

FR-2 - Safe AI Waiter Chat

- **User story:** As a diner with allergies or a budget, I want to ask the AI waiter in my language about allergens, vegetarian dishes, prices and opening hours, so that I get correct answers from the restaurant's own data.
- **Pain this traces to:** Diners cannot ask questions of a static menu, and a wrong chatbot answer about allergens creates real harm and legal liability.
- Given a question that names an allergen or asks about a dish's allergens, when it is submitted, then the reply lists dishes from the allergen data, names the dishes whose allergen data is missing, ends with the fixed "please confirm with the staff" sentence, never says a dish is safe, and appears within 1 second.
- Given an open question (recommendation, FAQ, small talk), when it is submitted, then the reply is in the language of the question within 15 seconds, and any model reply that claims safety, contains a price that differs from the menu, or is empty is replaced by "please ask the staff".
- Given a message that tries to make the assistant ignore its rules, when it is submitted, then the assistant refuses with a fixed sentence and changes no data.

FR-3 - Picks, Orders and Staff Calls

- **User story:** As a diner, I want to collect the dishes I like and send them to the waiter, and call the staff from my phone, so that ordering is quick and the waiter knows about my allergy.
- **Pain this traces to:** Waiting for staff to notice a table; allergy details lost between the diner and the kitchen.

- Given a diner has at least one available dish in My picks, when "Send to staff" is tapped, then an order with status picked appears on the waiter screen within 5 seconds, including any allergen the diner mentioned in the chat; sold-out dishes are left out and an order with no available dish is rejected with a message.
- Given the diner presses "Call staff" or asks for the bill twice for the same table, when the calls are saved, then only one open call of that type is shown to the waiter.

FR-4 - Waiter and Kitchen Live Screens

- **User story:** As a waiter or chef, I want live screens for calls, picks and kitchen tickets, so that no call or allergy warning is missed and dishes move from order to table in the right order.
- **Pain this traces to:** Staff shortages and order errors; missed allergy notes between waiter and kitchen.
- Given a picked order, when the waiter taps "Take order", then a New ticket with quantities and an allergy banner appears on the chef screen within 5 seconds; the order can only move picked to new to cooking to ready to served, and a wrong role or a skipped step is rejected by the server (HTTP 403 or 409).
- Given the chef or waiter marks a dish sold out, when the change is saved, then the diner menu shows it within 20 seconds and the AI waiter reports it sold out immediately.

FR-5 - Owner Account and Menu / Allergen Management

- **User story:** As a restaurant owner, I want to sign in and manage dishes, categories, prices, photos, tags and allergens in three languages, so that diners and the AI waiter always use correct information.
- **Pain this traces to:** Owners abandon tools that are costly or hard to keep up to date; allergen data must be owned by the restaurant.
- Given valid email, a password of at least 8 characters and a restaurant name, when the owner registers, then an account, a unique restaurant code and four default categories are created; a wrong password is rejected.
- Given a dish with a name, a price of 0 or more and allergens (or "not provided"), when the owner saves, then it appears on the diner menu within 20 seconds; a blank name or negative price is rejected with a message, and an owner cannot read or edit another restaurant's dishes (HTTP 404).

FR-6 - Menu Import from a Photo

- **User story:** As a restaurant owner, I want to photograph my printed menu and have the AI read it, so that I can publish my menu in minutes instead of typing it.
- **Pain this traces to:** About 70% of restaurant menus exist only as PDFs or print; setup effort is a main reason owners abandon tools.
- Given a JPG, PNG or WebP photo under 8 MB, when the owner uploads it, then dish names, prices and categories are shown for review within 120 seconds; rows with a price of 0 or digits inside the name are flagged; any other file type or size is rejected with a message.

- Given the review list, when the owner has not tapped "Confirm and publish", then no dish is added to the live menu; after confirming, all imported dishes have allergen data "not provided".

FR-7 - Restaurant Settings (Hours, Specials, FAQ, AI Voice)

- **User story:** As a restaurant owner, I want to set opening hours, specials, FAQ answers, shop rules and the voice of my AI waiter, so that the assistant answers like my restaurant.
- **Pain this traces to:** Diners ask the same practical questions (hours, Wi-Fi, parking) and expect the AI to know the shop's own rules.
- Given opening, closing and last-order times in HH:MM, when the owner saves, then the AI quotes exactly those times in its next answer; an invalid time is rejected.
- Given the owner selects a female voice, when a diner asks in Thai or Burmese, then the reply ends with "ค่ะ" or "ရှင်"; with the default male voice it ends with "ครับ" or "ခင်ဗျာ".

FR-8 - QR Codes per Table

- **User story:** As a restaurant owner, I want to print a QR code for every table, so that diners reach the menu of the right table without typing an address.
- **Pain this traces to:** Diners will not type addresses; the waiter must know which table called.
- Given a number of tables from 1 to 60 and a base address, when the owner opens the QR page, then one printable QR code per table (SVG) is shown, each opening the menu with that table number.
- Given the owner picks the server's Wi-Fi address, when a phone on the same Wi-Fi scans a code, then the diner menu opens with the table number shown.

FR-9 - Staff Accounts (PIN)

- **User story:** As a restaurant owner, I want to give my waiters and chefs a PIN, so that each of them sees only their own screen without email accounts.
- **Pain this traces to:** High staff turnover makes full accounts impractical; a shared login exposes owner data.
- Given a name, a role (waiter or chef) and a PIN of 4 to 8 digits, when the owner saves, then the staff member can sign in with the restaurant code and PIN and is taken to the screen of their role; a PIN of fewer than 4 digits is rejected and PINs are stored hashed.
- Given a wrong PIN, when a staff member signs in, then access is refused and no data is shown.

FR-10 - Insights

- **User story:** As a restaurant owner, I want to see what diners asked and what the AI could not answer, so that I can add missing dishes, FAQs and information.
- **Pain this traces to:** Owners see orders but not unmet demand, for example many diners asking for vegan food.
- Given a period of 1, 7 or 30 days, when the owner opens Insights, then it shows the number of chats, the share answered from data, the most-asked topics, the languages used and the unanswered questions with an "Add FAQ" shortcut.
- Given diners asked vegetarian or vegan questions, when the owner has fewer vegetarian dishes than questions, then an "unmet demand" card is shown; only the owner's own restaurant data is included.

4. Non-functional requirements

Forbidden words unless quantified: fast, easy, user-friendly, smart, secure. NFR-5 (security/privacy) is required for M2 — every SRS must have at least one.

ID	Kind	Statement (testable)	How we will check
NFR-1	performance	The AI waiter shall reply to open questions within 15 seconds for 90% of questions on the reference computer (MacBook Air M4, 32 GB, gemma4:12b). Rule-based answers (allergens, prices, hours, orders) shall reply within 1 second.	Response times logged over the 30-question test set and 100 open questions: 100 of 100 within 15 s (median 5.5 s, 90th percentile 10.7 s, idle laptop).
NFR-2	performance	The diner menu shall show dishes within 3 seconds on the local network, and staff screens shall refresh at least every 3 seconds.	Server side measured: page 50 ms, menu 15 ms. Phone over Wi-Fi: still to measure.
NFR-3	usability	A first-time restaurant owner shall publish a first menu using photo import in under 30 minutes without help.	Timed trial with 3 restaurant owners or students playing owners (target, still to do).
NFR-4	reliability	If the AI model is unavailable, the diner shall still browse the menu, send picks and call staff, and the chat shall show a fallback message within 5 seconds.	Stop Ollama, then repeat the diner flow and check every step (passed in the e2e test).
NFR-5	security / privacy (required)	A user shall access only their own restaurant's data; passwords and PINs shall be stored with bcrypt; the system shall collect no diner name, phone or email; and no data shall be sent to a cloud AI service.	API test that another restaurant's dish returns 404 (passed), inspection of the database for password hashes, and a network capture showing traffic only to localhost:11434 for AI.
NFR-6	safety	Across the 30-question test set and the rule-change test, 0 replies shall say a dish is safe or allergy-free, and 100% of allergen replies shall end with the "confirm with the staff" sentence.	Automated chat test (scripts/chat_smoke.py): 30 of 30 passed.
NFR-7	capacity	The system shall serve 3 diners chatting at the same time and 2 staff screens without an error, each AI answer within 30 seconds	Simulated: 3 diners, 2 staff screens, 10 min: 0 errors, slowest AI answer 17.4 s.

5. Use cases

5.1 Use-case list

Use case	Actor	Related FR	Brief success story
UC-1 Browse the menu	Diner	FR-1	Diner scans the table QR code, picks Burmese, filters "No peanuts" and opens a dish.
UC-2 Ask the AI waiter	Diner	FR-2	Diner types "I'm allergic to peanuts"; the AI lists the dishes with peanut and the dishes without allergen data, and asks to confirm with the staff.
UC-3 Send picks or call staff	Diner	FR-3	Diner adds two dishes and taps "Send to staff"; the floor staff see the order with the allergy note.
UC-4 Take and cook an order	Waiter, Chef	FR-4	Waiter takes the order; chef starts cooking and marks it ready; waiter serves it.
UC-5 Manage menu and allergens	Owner	FR-5	Owner adds a dish, ticks its allergens and switches a sold-out dish off.
UC-6 Import a menu photo	Owner	FR-6	Owner photographs the printed menu, corrects two flagged rows and publishes 14 dishes.
UC-7 Set hours, FAQ and AI voice	Owner	FR-7	Owner sets closing time 22:00; the AI answers "ร้านปิดกี่โมง" with 22:00.
UC-8 Print QR codes	Owner	FR-8	Owner prints 8 table codes and a phone opens the menu for table 5.
UC-9 Manage staff	Owner	FR-9	Owner creates a chef with PIN 2222; the chef signs in and lands on the kitchen screen.
UC-10 Read insights	Owner	FR-10	Owner sees 47 vegetarian questions but only 2 vegetarian dishes and adds one.

5.2 Use-case diagram

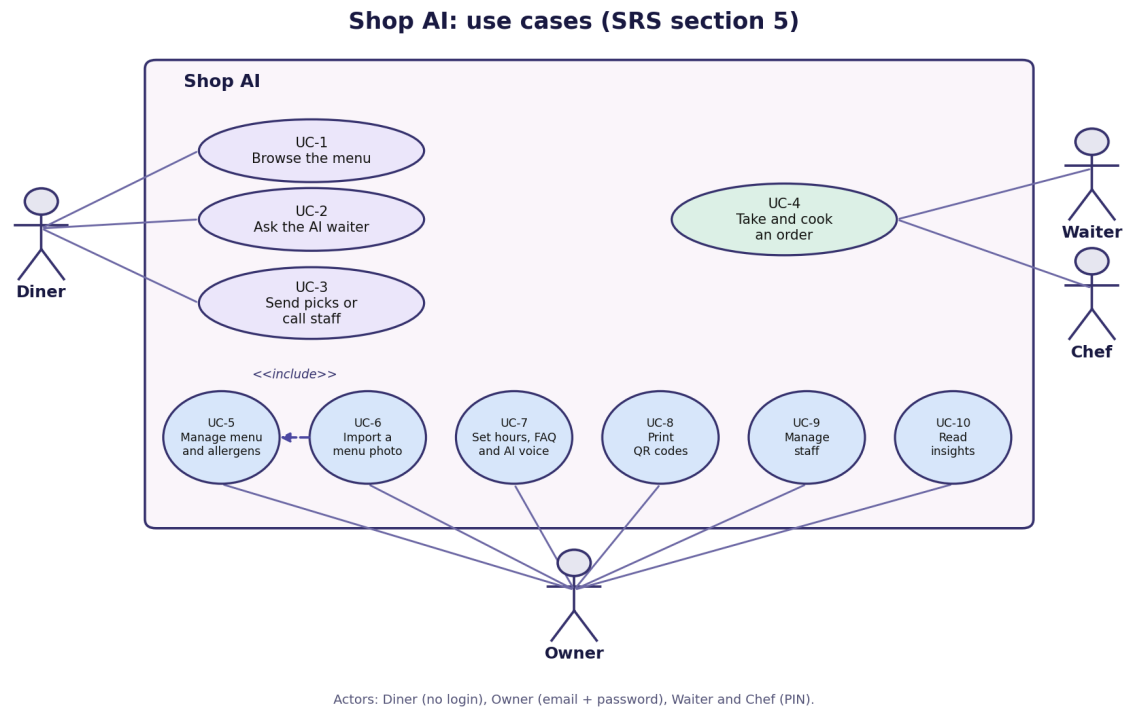
Figure 1 on the next page shows the use-case diagram. The PlantUML code it was drawn from follows.

```
@startuml
left to right direction
actor Diner
actor Waiter
actor Chef
actor Owner
rectangle "Shop AI" {
    usecase "UC-1 Browse the menu" as UC1
    usecase "UC-2 Ask the AI waiter" as UC2
    usecase "UC-3 Send picks or call staff" as UC3
    usecase "UC-4 Take and cook an order" as UC4
    usecase "UC-5 Manage menu and allergens" as UC5
    usecase "UC-6 Import a menu photo" as UC6
    usecase "UC-7 Set hours, FAQ and AI voice" as UC7
    usecase "UC-8 Print QR codes" as UC8
    usecase "UC-9 Manage staff" as UC9
    usecase "UC-10 Read insights" as UC10
}
Diner --> UC1
Diner --> UC2
Diner --> UC3
Waiter --> UC4
Chef --> UC4
Owner --> UC5
Owner --> UC6
Owner --> UC7
Owner --> UC8
Owner --> UC9
Owner --> UC10
UC6 ..> UC5
<> @enduml
```

6. Out of scope

1. Online payment, bill splitting and delivery.
2. LINE and WhatsApp channels, native mobile apps and POS integration.
3. Loyalty programmes and billing plans.
4. AI-generated dish photos and email summaries.
5. Order status shown to diners, and the owner dashboard in Thai or Burmese (planned for a later phase).

Figure 1. Use-case diagram (Section 5.2)



Actors: Diner, Waiter, Chef, Owner. UC-1 to UC-10 are the use cases of Section 5.1; UC-6 includes UC-5.

7. Traceability matrix (Golden Thread)

Every functional requirement is traced from the pain to the tested feature. Automated tests run on every push (see docs/TRACEABILITY.md in the repository). Tests marked "manual" are still to be run.

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
Hard-to-read QR menus, tourists cannot read the language	FR-1 Menu browsing	Mobile page with photos, filters, three-language names	Diner menu page /r/[code]	e2e: public menu returns 13 dishes; unknown restaurant returns 404; manual phone test
Allergy questions are risky and chatbots invent answers	FR-2 Safe AI chat	Rules answer from the database; model only for open questions; output checks	AI waiter chat with dish cards	chat test 30 of 30; e2e: allergy chat, sold-out state, Thai female voice, hours
Waiting for staff; allergy lost between diner and kitchen	FR-3 Picks, orders, calls	Orders carry the allergens mentioned in chat; calls de-duplicated	My picks, Send to staff, call-staff button	e2e: picks with sold-out dish, empty order rejected, duplicate call reused
Missed calls and order mistakes	FR-4 Waiter and kitchen screens	Order status flow with role and step checks; 3-second refresh	Waiter floor and chef kitchen pages	e2e: 20 checks for take, cook, ready, served, role limits and sold-out sync
Outdated menus and allergen data	FR-5 Owner account and menu management	Owner login, per-restaurant data, allergen or "not provided"	Owner menu items page	e2e: register, login, create, update, delete, validation, isolation
Typing a whole menu takes hours	FR-6 Photo import	Vision model reads the photo; owner reviews; nothing is live until confirmed	Import a menu page	e2e: photo import finds dishes and prices, confirm import, allergens unknown
AI must know the shop's own rules	FR-7 Settings	Hours, specials, FAQ, rules and voice fed to the AI	Specials and hours, FAQ and rules, AI waiter pages	e2e: new hours quoted by the AI, invalid time rejected, female voice
Diners cannot reach the right table	FR-8 QR codes	SVG QR per table with Wi-Fi address option	QR codes page	e2e: QR SVG generated; manual: scan with a phone
Staff turnover, shared logins	FR-9 Staff accounts	PIN sign-in with role, hashed PIN	Staff page, /staff sign-in	e2e: short PIN rejected, create staff, PIN hashes never listed, wrong PIN rejected
Owners cannot see unmet demand	FR-10 Insights	Chat log with topic, language and answered flag	Insights page	e2e: insights totals and topics; manual review with real chats

8. AI usage log

Drafted from the working session; the team must check each row and complete the last two columns for its own actions.

Date	Tool model /	What we asked	What AI produced	What we changed / verified	What we learned
21 Sep 2026	Claude Code (Claude Sonnet 5)	Turn the market research report into a feature plan and search Hugging Face for a suitable model.	Feature plan (P0/P1/P2), model shortlist (Qwen, SEA-LION, Gemma), sizing for a GPU server and then for the laptop.	Chose Thai, Burmese and English, and a MacBook Air M4 with 32 GB; models re-checked on the Hugging Face listing.	The first idea (27B models) did not fit the laptop; hardware limits must come first.
21 Sep 2026	Claude Code	Build a 30-question test set and compare four local models.	Test menu, 30 questions in 3 languages, an evaluation script and a results table.	Ran it on the laptop; read the failed answers; fixed the checker's false alarms; chose gemma4:12b.	Keyword checks give false alarms, and every model missed some allergen dishes, so allergen answers must come from the database.
21 Sep 2026	Claude Code	Show Burmese answers for review and apply corrections.	Four rounds of Burmese answers on a review web page and prompt changes.	Our Burmese speaker rated each answer and gave corrections (male ending "ခင်ဗျာ", "Allergens" in English, no formal "သည်").	AI cannot judge Burmese quality; a native speaker found a wrong closing time and stiff wording.
21 Sep 2026	Claude Code	Design the UI in Art Deco and Glass styles, plus waiter and chef screens.	A design canvas with two style pages and five to seven screens each.	We viewed both styles and chose Glass.	AI mock-ups are quick, but the team must pick the style and check readability.
21 Sep 2026	Claude Code	Build all UI features and the database.	A Next.js and SQLite application with 12 tables, about 30 API routes and all screens.	Ran a 64-check API test and the 30-question chat test; fixed bugs found (fish matched inside shellfish, "no shellfish" list reversed, quantity after a dish name). The screens have not yet been checked in a browser.	Tests found real bugs that the first version had; visual and phone testing are still needed.
21 Sep 2026	Claude Code	Convert our SRS into this course template.	This document.	[team to verify every FR, number and test result before submission]	[team to complete

8. AI usage log (continued)

Rows added after the first session. The team must check each row and complete the last columns.

Date	Tool / model	What we asked	What AI produced	What we changed / verified	What we learned
22–23 Sep 2026	Claude Code (Claude Sonnet 5)	Speed up the AI waiter; build accessible diner and staff screens, the "Ask AI" card and fixes.	A smaller prompt and a shorter history; UI code, CSS and copy changes (15 commits).	[team to complete]	[team to complete]
5 Oct 2026	Claude Code (Claude Sonnet 5.5)	Add automated tests, CI, Docker, a seed script and the documents TRACEABILITY, AI_SAFETY and AI_USAGE_LOG.	A Vitest suite and end-to-end tests, GitHub Actions, Dockerfile and compose, a seed script, the documents.	Ran lint, type-check, tests, build, Docker and the live eval after each change; also in a clean Linux container. [team to read the diff and confirm]	Writing tests found real bugs (the bill keyword matching inside "papaya"; a wrong quantity for the second dish in "x4 ... x2"; an unchecked price before a dish name; "database is locked" on the first build).
5 Oct 2026 (later)	Claude Code (Claude Sonnet 5.5)	Cover all 14 allergens; rate-limit chat, login and register; check dish names and prices in model replies; check upload contents; unique PINs; load tests.	Keyword lists, a rate limiter, new reply checks, an image-type check, a load-test script and a performance document.	Measured the new checks against 34 real model answers (0 wrongly replaced); load-tested 3 and 30 diners, and 10 minutes with staff screens. [team to confirm]	A local model is slower when run in parallel (3 at once took 32 s instead of 15 s); the SRS already said "one at a time" but the code did not enforce it.
5 Oct 2026 (merge)	Claude Code (Claude Sonnet 5.5)	Merge two parallel lines of AI-assisted work (SRS-gap fixes with local-only AI, and the tests, CI and hardening work).	A merge with 18 conflicts resolved by hand; then the allergy profile, recommendations and owner insights were merged in.	Lint, type-check, all tests, the build and the end-to-end tests pass on the merged branch. [team to review the resolutions]	Two sessions working in parallel on one repository duplicate each other; the decisions about which version to keep must be made by people.
5 Oct 2026 (security)	Claude Code (Claude Sonnet 5.5)	Fix seven security findings: PIN lockout bypass, the chat preview flag, cookie flags, login timing, the receipt code, chat history scope, and the docs.	A per-restaurant PIN cap, owner-only preview, Secure cookies, async bcrypt, a server-issued receipt code, and tests for each.	Each fix was checked by reverting it and seeing its test fail; lint, type-check, all tests, the build and the end-to-end tests pass. [team to review]	A first version of one test passed even with the bug (the model sees only the last two messages); reverting the fix is how it was caught.
5 Oct 2026 (M4)	Claude Code (Claude Sonnet 5.5)	Prepare the Milestone 4 deliverables: a clickable prototype without a server, a final report draft, demo slides and an AI-use statement.	The prototype (diner, staff and owner journeys with mock data), a test that ties it to this SRS, the report draft, the slides.	Clicked through in a real browser; a test checks it cites only real SRS IDs and mentions every Must requirement. [team to review]	A test that maps every Must requirement to a screen found one (AI-16) that no screen mentioned.

9. Self-check

[x] **Every FR is a user story with a benefit** - All 10 FRs in Section 3 end in a "so that" clause.

[x] **Every FR has acceptance criteria you could actually test** - Each FR has 2 or more Given/When/Then criteria with numbers (for example 3 s menu, 1 s rule answer, 15 s AI answer, 5 s order to kitchen).

[] **Every FR traces back to a stated problem from a real stakeholder** - The pains come from published surveys and three M1 interviews; the FRs are not yet checked with them.

[x] **NFRs exist, including security / privacy** - NFR-5 covers own-data access, hashed passwords, no personal data and no cloud AI.

[x] **Scope and out-of-scope are stated** - Section 2.1 and Section 6.

[x] **Use-case list matches the FR list** - 10 use cases map 1:1 to the 10 FRs in Section 5.1.

[x] **Use-case diagram inserted** - The diagram is in Section 5.2 (Figure 1, next page).

[] **AI log is honest** - Drafted from the working session; the team must confirm it and complete the last columns.

[] **Requirements checked with a real stakeholder** - Only the AI's Burmese answers were reviewed by a native speaker; the M1 interviews were about the problem, not these requirements.

Course rule: you may use AI to draft. You must log it, verify it against a real stakeholder, and be able to explain every FR in the oral exam. Work you cannot explain earns no credit.